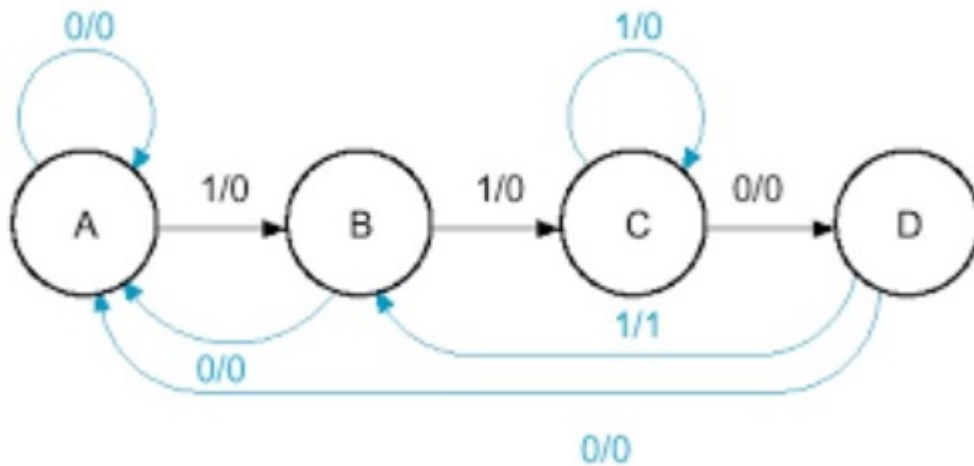


Vibe Coding mô phỏng mạch tuần tự

Prof Happy AI-Suu tâm và biên soạn với sự hỗ trợ của Gemini

Date: May 4, 2026

- ✓ Vibe Coding tạo Code mô phỏng mạch số có state diagram



```

class MealyFSM:
    def __init__(self):
        # Khởi tạo trạng thái ban đầu là A
        self.state = 'A'

    def transition(self, bit):
        """
        Xử lý sự thay đổi trạng thái và trả về đầu ra dựa trên sơ đồ:
        Ký hiệu trên cung: input / output
        """
        output = 0
        current_state = self.state

        if self.state == 'A':
            if bit == 1:
                self.state = 'B'
                output = 0
            else:
                self.state = 'A'

```

```

        output = 0

    elif self.state == 'B':
        if bit == 1:
            self.state = 'C'
            output = 0
        else:
            self.state = 'A' # Cung từ B về A khi input=0
            output = 0

    elif self.state == 'C':
        if bit == 1:
            self.state = 'C' # Loop tại C khi input=1
            output = 1      # Lưu ý: Cung ghi 1/1
        else:
            self.state = 'D'
            output = 0

    elif self.state == 'D':
        if bit == 1:
            self.state = 'B' # Cung từ D về B khi input=1
            output = 0
        else:
            self.state = 'A' # Cung từ D về A khi input=0
            output = 0

    return current_state, output

# --- CHƯƠNG TRÌNH CHÍNH ---
fsm = MealyFSM()
test_sequence = [1, 1, 0, 1, 1, 0, 0, 1]

print(f"{'Input':<8} | {'Current State':<15} | {'Next State':<12} | {'Output':<6}")
print("-" * 55)

for bit in test_sequence:
    old_state, out = fsm.transition(bit)
    print(f"{bit:<8} | {old_state:<15} | {fsm.state:<12} | {out:<6}")

```

Input	Current State	Next State	Output
1	A	B	0
1	B	C	0
0	C	D	0
1	D	B	0
1	B	C	0
0	C	D	0
0	D	A	0
1	A	B	0

Dựa trên hình ảnh bạn vừa tải lên, đây là một Máy trạng thái hữu hạn (FSM) kiểu Mealy (vì đầu ra phụ thuộc vào cả trạng thái hiện tại và tín hiệu đầu vào, ký hiệu dạng input/output).

Mạch này có 4 trạng thái: A, B, C, D. Mục tiêu của mạch có vẻ là nhận diện chuỗi bit 1101 (khi đạt đến trạng thái C và nhận đầu vào 1, đầu ra sẽ nhảy lên 1). Giải thích logic chuyển trạng thái từ hình ảnh: Trạng thái A: Nếu vào 1 -> sang B, nếu vào 0 -> ở lại A.

Trạng thái B: Nếu vào 1 -> sang C, nếu vào 0 -> quay về A.

Trạng thái C:

Nếu vào 0 -> sang D (output 0).

Nếu vào 1 -> ở lại C (đầu ra output = 1). Đây là điểm đặc biệt nhất trong sơ đồ của bạn.

Trạng thái D: Nếu vào 1 -> quay lại B, nếu vào 0 -> quay về A.

Verilog

✓ design.v module

```
module mealy_fsm ( input clk, input reset, input x, // Tín hiệu đầu vào output reg z //
Tín hiệu đầu ra );
```

```
// Định nghĩa các trạng thái
parameter A = 2'b00,
          B = 2'b01,
          C = 2'b10,
          D = 2'b11;

reg [1:0] current_state, next_state;

// 1. Bộ nhớ trạng thái (Sequential Logic)
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= A;
    else
        current_state <= next_state;
end

// 2. Logic chuyển trạng thái và đầu ra (Combinational Logic)
always @(*) begin
    // Mặc định tránh tạo Latch
    next_state = current_state;
    z = 1'b0;

    case (current_state)
        A: begin
            if (x) next_state = B;
        end
    end
```

```

        else    next_state = A;
        z = 1'b0;
    end
B: begin
    if (x) next_state = C;
    else    next_state = A;
    z = 1'b0;
end
C: begin
    if (x) begin
        next_state = C;
        z = 1'b1; // Cung 1/1 tại trạng thái C
    end else begin
        next_state = D;
        z = 1'b0;
    end
end
D: begin
    if (x) next_state = B;
    else    next_state = A;
    z = 1'b0;
end
default: next_state = A;
endcase
end

```

endmodule

testbench.v module

module testbench; reg clk; reg reset; reg x; wire z;

```

// Khởi tạo module FSM
mealy_fsm uut (
    .clk(clk),
    .reset(reset),
    .x(x),
    .z(z)
);

// Tạo xung Clock (chu kỳ 10 đơn vị thời gian)
always #5 clk = ~clk;

initial begin
    // Khởi tạo tín hiệu
    clk = 0;

```

```
reset = 1;
x = 0;

// Thả reset sau 15 đơn vị thời gian
#15 reset = 0;

// Test sequence dựa trên sơ đồ:
// Mục tiêu: Đi từ A -> B -> C -> C (để thấy z=1) -> D -> B

#10 x = 1; // A -> B
#10 x = 1; // B -> C
#10 x = 1; // C -> C (Z nên bằng 1 ở đây)
#10 x = 0; // C -> D
#10 x = 1; // D -> B
#10 x = 0; // B -> A

#20;
$display("Ket thuc mo phong.");
$finish;
end

// Hiển thị kết quả trên Console
initial begin
    $monitor("Time=%0t | State=%b | Input x=%b | Output z=%b", $time, uut.current, x, z);
end

// Để xem dạng sóng trên EDA Playground
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1);
end
```

endmodule

Giao diện edaplayground.com

The screenshot displays the EDA Playground web interface. The top navigation bar includes 'Edit code', 'EDA Playground', and 'Hypervisors in the Real World'. The main workspace is divided into three panels:

- Left Panel:** Contains project settings. Under 'Languages & Libraries', 'SystemVerilog/Verilog' is selected. Under 'Tools & Simulators', 'Icarus Verilog 12.0' is chosen. The 'Run Options' section is expanded, showing checkboxes for 'Use run.bash shell script', 'Open ESWave after run', 'Show output test file', 'Show output SVG file', and 'Download files after run'. The 'Examples' section lists various templates like 'using EDA Playground', 'VHDL', 'Verilog/SystemVerilog', 'UVM', 'EasierUVM', 'SVAUnit', 'SVUnit', 'VUnit (Verilog/SV)', and 'VUnit (VHDL)'.
- Top Right Panel:** Displays two Verilog code files. The left file, 'testbench.v', contains a testbench for a 3-bit counter with inputs A, B, and C, and outputs x, y, and z. It includes comments in Vietnamese and a \$monitor statement. The right file, 'design.v', contains the logic for a 3-bit counter with three parallel counters (A, B, and C) and a final output z.
- Bottom Panel:** Shows the simulation log. It includes a timestamp '2026-05-04 09:18:00 UTC', the command 'verilog -Wall -g2012 design.v testbench.v -A unbuffer -vvp &.out', and the output of the Icarus Verilog compiler. The log shows a warning about the \$dumpvars statement and a successful compilation. The simulation results show the state of the circuit at various time intervals (Time=0, Time=25, Time=35, Time=55, Time=65) and the final state of the outputs.